

POSTGRESQL

Backup & Disaster Recovery

Lab Manual

pgBackRest — Configure • Full • Incremental • Differential • Restore • PITR

Lab Environment

Kubernetes Cluster | Namespace: **postgres-ha**
Cluster: **acid-postgres-ha** (Zalando Postgres Operator / Spilo)
Pods: acid-postgres-ha-0 (primary), -1, -2 | Pooler + Exporter
Tool: **pgBackRest** | OS: Ubuntu 24.04

Generated: 24 September 2026
Duration: ~90–120 minutes | Level: Intermediate

Table of Contents

1. Lab Objectives & Prerequisites
2. Environment Overview
3. LAB 1 — Configure pgBackRest
4. LAB 2 — Full Backup
5. LAB 3 — Incremental Backup
6. LAB 4 — Differential Backup
7. LAB 5 — Restore Database
8. LAB 6 — Point-in-Time Recovery (PITR)
9. Verification Checklist
10. Troubleshooting Guide
11. Appendix — Useful Commands & Retention

1. Lab Objectives & Prerequisites

Objectives

By the end of this lab you will be able to:

- Install and configure pgBackRest for a PostgreSQL cluster (stanza, repository, archive_command).
- Perform a full physical backup and verify its integrity.
- Create incremental and differential backups and understand the difference.
- Restore a cluster from a pgBackRest backup.
- Execute Point-in-Time Recovery (PITR) to a specific timestamp.
- Apply retention policies and monitor backup health.

Prerequisites

- Access to the Kubernetes cluster with kubectl configured (namespace: postgres-ha).
- Cluster acid-postgres-ha running (pods -0, -1, -2 Ready).
- Ability to exec into the primary pod (or a dedicated backup host).
- Basic familiarity with PostgreSQL and Linux shell.
- Optional: S3-compatible storage (MinIO / AWS S3) for production-like repository.

Lab Assumptions

These exercises can be performed either (a) by installing pgBackRest inside the primary Spilo pod for learning purposes, or (b) on a dedicated backup host / sidecar that can reach the primary via the replication protocol. Commands shown use a local repository path for simplicity; replace with S3/GCS settings for production.

2. Environment Overview

Your current Kubernetes environment (from `kubectl get pods -n postgres-ha`):

Pod	Role	Ready
acid-postgres-ha-0	Primary (Patroni leader)	1/1
acid-postgres-ha-1	Replica	1/1
acid-postgres-ha-2	Replica	1/1
acid-postgres-ha-pooler-*	PgBouncer connection pooler	1/1
postgres-exporter-...	Prometheus metrics exporter	1/1

Zalando Postgres Operator (Spilo) normally uses WAL-G for continuous archiving. In this lab we introduce **pgBackRest** as the enterprise-grade physical backup tool that supports full, differential and incremental backups plus native PITR.

Key Concepts Recap

- Full** — complete copy of all data files.
- Differential** — changes since the last *full* backup.
- Incremental** — changes since the last backup of any type (full, diff or incr).
- PITR** — restore base backup + replay WAL to a chosen timestamp.

LAB 1 — Configure pgBackRest

Goal: Install pgBackRest (if needed), create the configuration file, initialize a stanza, and enable WAL archiving so that continuous archiving works.

Step 1.1 — Identify the Primary Pod

```
# From your workstation
kubectl get pods -n postgres-ha -l application=spilo

# Find the current leader (Patroni)
kubectl exec -n postgres-ha acid-postgres-ha-0 -c postgres -- \
  patronictl list

# Or use the operator label / role annotation
kubectl get pods -n postgres-ha -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.metadata.labels.spilo-role}
```

Step 1.2 — Enter the Primary Pod (or Backup Host)

```
kubectl exec -it -n postgres-ha acid-postgres-ha-0 -c postgres -- bash
# You should now be inside the Spilo container as root/postgres
```

Step 1.3 — Install pgBackRest (if not present)

```
# Check whether pgBackRest is already available
which pgbackrest || echo "not found"

# On Debian/Ubuntu-based Spilo images you may need to install:
apt-get update && apt-get install -y pgbackrest
# (In production prefer a sidecar or dedicated backup host)
```

Step 1.4 — Create Repository Directory

```
mkdir -p /var/lib/pgbackrest
chmod 750 /var/lib/pgbackrest
chown postgres:postgres /var/lib/pgbackrest

# Also create log & spool directories
mkdir -p /var/log/pgbackrest /var/spool/pgbackrest
chown postgres:postgres /var/log/pgbackrest /var/spool/pgbackrest
```

Step 1.5 — Write pgBackRest Configuration

Create /etc/pgbackrest/pgbackrest.conf (or /etc/pgbackrest.conf):

```
[global]
repo1-path=/var/lib/pgbackrest
repo1-retention-full=2
repo1-retention-diff=7
repo1-retention-archive=14
start-fast=y
process-max=2
compress-type=zst
compress-level=3
log-level-console=info
log-level-file=detail
log-path=/var/log/pgbackrest

[global:archive-push]
compress-type=zst

[acid-ha] # stanza name
pg1-path=/home/postgres/pgdata/pgroot/data
pg1-port=5432
pg1-user=postgres
# For Spilo the data directory is typically under /home/postgres/pgdata/pgroot/data
# Confirm with: echo $PGDATA or psql -c 'SHOW data_directory'
```

Spilo / Zalando Note

Inside Spilo the PostgreSQL data directory is usually /home/postgres/pgdata/pgroot/data. Always verify with `psql -c 'SHOW data_directory;'` and adjust `pg1-path` accordingly.

Step 1.6 — Create the Stanza

```
sudo -u postgres pgbackrest --stanza=acid-ha stanza-create

# Verify
sudo -u postgres pgbackrest --stanza=acid-ha check
```

Step 1.7 — Enable WAL Archiving in PostgreSQL

Configure `archive_command` so PostgreSQL hands completed WAL segments to pgBackRest:

```
# Connect as superuser
psql -U postgres

-- Check current settings
SHOW wal_level;
SHOW archive_mode;
SHOW archive_command;

-- Required (may already be set by Spilo/Patroni)
-- wal_level must be 'replica' or higher
-- archive_mode = on (requires restart if changing from off)

-- Set archive_command (can be done via ALTER SYSTEM or Patroni config)
ALTER SYSTEM SET archive_command =
    'pgbackrest --stanza=acid-ha archive-push %p';
SELECT pg_reload_conf();

-- Force a WAL switch and confirm archiving
SELECT pg_switch_wal();
SELECT * FROM pg_stat_archiver;
```

In a Zalando-managed cluster you normally change archive settings through the Postgresql CRD or Patroni dynamic configuration rather than ALTER SYSTEM. For the lab, ALTER SYSTEM + reload is acceptable on a non-production test cluster.

Verification — Lab 1

```
sudo -u postgres pgbackrest --stanza=acid-ha check
# Expected: "check command end: completed successfully"

psql -U postgres -c "SELECT archived_count, last_archived_wal, last_failed_wal FROM pg_stat_archiver;"
```

LAB 2 — Full Backup

Goal: Take a complete physical backup of the cluster and inspect the repository.

Step 2.1 — Run a Full Backup

```
sudo -u postgres pgbackrest --stanza=acid-ha --type=full \  
    --log-level-console=info backup
```

Step 2.2 — Inspect Backup Info

```
sudo -u postgres pgbackrest --stanza=acid-ha info  
  
# More detail  
sudo -u postgres pgbackrest --stanza=acid-ha info --output=json | head -c 2000
```

Step 2.3 — List Files in the Repository

```
ls -lh /var/lib/pgbackrest/backup/acid-ha/  
du -sh /var/lib/pgbackrest/
```

What a Full Backup Contains

A full backup copies every file in the PostgreSQL data directory that is needed to restore the cluster. Combined with subsequent WAL, it is the foundation for PITR. Size is approximately equal to the size of the data directory (before compression).

Verification — Lab 2

```
sudo -u postgres pgbackrest --stanza=acid-ha info  
# Look for: status: ok    and a backup labeled 'full' with a recent timestamp
```

LAB 3 — Incremental Backup

Goal: Generate some data changes, take an incremental backup, and observe that only changed blocks are stored.

Step 3.1 — Generate Change (Simulate Workload)

```
psql -U postgres <<'SQL'
CREATE TABLE IF NOT EXISTS lab_demo (
  id    serial PRIMARY KEY,
  payload text,
  created_at timestamptz DEFAULT now()
);
INSERT INTO lab_demo (payload)
SELECT md5(g::text) FROM generate_series(1, 10000) g;
SELECT count(*) FROM lab_demo;
SQL
```

Step 3.2 — Take Incremental Backup

```
sudo -u postgres pgbackrest --stanza=acid-ha --type=incr \
  --log-level-console=info backup
```

Step 3.3 — Compare Sizes

```
sudo -u postgres pgbackrest --stanza=acid-ha info
# Notice the incremental backup is much smaller than the full backup
```

Incremental vs Differential

Incremental stores changes since the *previous* backup of any type (full → incr → incr ...). Restore requires the full + every incremental in the chain.

Differential stores changes since the last *full* only. Restore = full + one diff.

LAB 4 — Differential Backup

Goal: Create additional changes and take a differential backup.

Step 4.1 — More Data Changes

```
psql -U postgres <<'SQL'
INSERT INTO lab_demo (payload)
SELECT md5((g+10000)::text) FROM generate_series(1, 5000) g;
UPDATE lab_demo SET payload = payload || '-upd' WHERE id <= 1000;
SELECT count(*) FROM lab_demo;
SQL
```

Step 4.2 — Take Differential Backup

```
sudo -u postgres pgbackrest --stanza=acid-ha --type=diff \
--log-level-console=info backup
```

Step 4.3 — Review Backup Set

```
sudo -u postgres pgbackrest --stanza=acid-ha info
# You should now see: 1 full + 1 incr + 1 diff (or similar)
```

Typical Production Schedule

Frequency	Type	Command
Weekly (Sun 01:00)	Full	pgbackrest --type=full backup
Daily	Differential	pgbackrest --type=diff backup
Every 4–6 h	Incremental	pgbackrest --type=incr backup
Continuous	WAL archive	archive-push via archive_command

LAB 5 — Restore Database

Goal: Perform a full restore of the cluster from the latest backup set. Because this is destructive, we restore into an alternate directory first.

■ **WARNING:** Restoring over a running primary will destroy current data. Always stop PostgreSQL and use a separate data directory (or a clone cluster) for practice restores.

Step 5.1 — Stop PostgreSQL (Lab / Test Only)

```
# Inside the pod - for a real Spilo/Patroni cluster you would
# scale down or create a new clone cluster instead.
# For this lab we demonstrate the pgBackRest restore command itself.

# Record current data directory
psql -U postgres -c "SHOW data_directory;"
```

Step 5.2 — Restore to an Alternate Path (Safe Practice)

```
export RESTORE_PATH=/tmp/pg_restore_lab
mkdir -p $RESTORE_PATH
chown postgres:postgres $RESTORE_PATH

sudo -u postgres pgbackrest --stanza=acid-ha \
  --type=immediate \
  --target-action=promote \
  --pg1-path=$RESTORE_PATH \
  --log-level-console=info \
  restore
```

Step 5.3 — Inspect Restored Files

```
ls -la /tmp/pg_restore_lab/
ls /tmp/pg_restore_lab/pg_wal/ | head
cat /tmp/pg_restore_lab/backup_label 2>/dev/null || true
# In PG 12+ look for recovery.signal or standby.signal
```

Step 5.4 — (Optional) Start a Temporary Instance on the Restored Data

```
# Only for lab verification - do NOT do this against the live cluster data dir
sudo -u postgres postgres -D /tmp/pg_restore_lab -p 5433 &
sleep 3
psql -h /tmp -p 5433 -U postgres -c "SELECT count(*) FROM lab_demo;"
# Stop when finished
pg_ctl -D /tmp/pg_restore_lab stop -m fast 2>/dev/null || true
```

Verification — Lab 5

Confirm that the restored data directory contains a valid PostgreSQL cluster structure and that the lab_demo table row count matches the state at the time of the backup.

LAB 6 — Point-in-Time Recovery (PITR)

Goal: Restore the cluster to a specific moment in time — after some desired data existed but before an accidental destructive change.

Step 6.1 — Capture a Target Timestamp

```
psql -U postgres <<'SQL'
-- Record the time BEFORE the "bad" change
SELECT now() AS target_time;
-- Insert a marker row we want to keep
INSERT INTO lab_demo (payload) VALUES ('KEEP-THIS-ROW');
SELECT id, payload, created_at FROM lab_demo WHERE payload = 'KEEP-THIS-ROW';
SQL

# Save the timestamp shown by now() e.g. 2026-09-25 05:30:00.123456+00
export TARGET_TIME="2026-09-25 05:30:00+00" # ← replace with your value
```

Step 6.2 — Simulate a Destructive Change

```
psql -U postgres <<'SQL'
-- "Oops" - accidental delete
DELETE FROM lab_demo WHERE id > 0;
SELECT count(*) FROM lab_demo; -- should be 0
SQL

# Force WAL switch so the delete is archived
psql -U postgres -c "SELECT pg_switch_wal();"
```

Step 6.3 — Perform PITR Restore

```
export RESTORE_PATH=/tmp/pg_pitr_lab
rm -rf $RESTORE_PATH
mkdir -p $RESTORE_PATH
chown postgres:postgres $RESTORE_PATH

sudo -u postgres pgbackrest --stanza=acid-ha \
  --type=time \
  --target="${TARGET_TIME}" \
  --target-action=promote \
  --pg1-path=$RESTORE_PATH \
  --log-level-console=info \
  restore
```

Step 6.4 — Verify PITR Result

```
# Start temporary instance on restored data
sudo -u postgres postgres -D /tmp/pg_pitr_lab -p 5434 &
sleep 4

psql -h /tmp -p 5434 -U postgres -c "SELECT count(*) FROM lab_demo;"
psql -h /tmp -p 5434 -U postgres -c \
  "SELECT id, payload, created_at FROM lab_demo WHERE payload = 'KEEP-THIS-ROW';"

# The KEEP-THIS-ROW should be present; the later DELETES should be absent.

pg_ctl -D /tmp/pg_pitr_lab stop -m fast 2>/dev/null || true
```

PITR Recovery Targets

pgBackRest supports several `--type` values for restore:

- **immediate** — end of the backup (no WAL replay beyond backup)
- **time** — `--target=TIMESTAMP`
- **xid** — `--target=TRANSACTION_ID`
- **name** — `--target=RESTORE_POINT_NAME` (from `pg_create_restore_point`)
- **lsn** — `--target=LSN`

9. Verification Checklist

- **Lab 1** — pgbackrest check succeeds; pg_stat_archiver shows archived_count increasing
- **Lab 2** — Full backup appears in pgbackrest info with status ok
- **Lab 3** — Incremental backup is present and smaller than the full
- **Lab 4** — Differential backup is present; info shows full + incr + diff
- **Lab 5** — Restore to alternate path produces a valid data directory
- **Lab 6** — PITR restores data up to (but not beyond) the chosen timestamp

After completing all labs, clean up temporary restore directories:

```
rm -rf /tmp/pg_restore_lab /tmp/pg_pitr_lab
```

10. Troubleshooting Guide

- **stanza-create fails / “unable to find primary”**

Confirm pg1-path points to the real data directory. Check PostgreSQL is running and pg1-user has sufficient privileges. Run as the postgres OS user.

- **archive_command fails / WAL accumulates in pg_wal**

Inspect pg_stat_archiver (last_failed_wal, failed_count). Test the command manually: pgbackrest --stanza=acid-ha archive-push . Check permissions on repo path.

- **Backup is very slow**

Increase process-max (CPU cores). Use compress-type=lz4 or zst. Backup from a standby to offload the primary (pg1-host / backup-standby options).

- **Restore complains about missing WAL**

Ensure continuous archiving was working between the base backup and the recovery target. Check repo archive directory for the required WAL segments.

- **Permission denied on repo**

Repository directory must be owned by the OS user that runs pgBackRest (usually postgres). chmod 750 is typical.

- **Spilo / Patroni overrides archive_command**

In Zalando Operator, set archive settings via the Postgresql CRD or Patroni configuration (postgresql.parameters) rather than only ALTER SYSTEM, otherwise they may be reset on restart.

11. Appendix — Useful Commands & Retention

Quick Reference Commands

```
# Stanza & health
pgbackrest --stanza=acid-ha stanza-create
pgbackrest --stanza=acid-ha check
pgbackrest --stanza=acid-ha info

# Backups
pgbackrest --stanza=acid-ha --type=full backup
pgbackrest --stanza=acid-ha --type=diff backup
pgbackrest --stanza=acid-ha --type=incr backup

# Expire old backups according to retention
pgbackrest --stanza=acid-ha expire

# Restore (latest)
pgbackrest --stanza=acid-ha --type=immediate --pg1-path=/new/data restore

# PITR to time
pgbackrest --stanza=acid-ha --type=time --target="2026-09-25 12:00:00" \
  --target-action=promote --pg1-path=/new/data restore
```

Retention Settings (in pgbackrest.conf)

```
repo1-retention-full=2      # keep 2 full backups
repo1-retention-diff=7      # keep 7 differential backups
repo1-retention-archive=14  # keep 14 days of WAL
repo1-retention-archive-type=full # or 'time'
```

S3 Repository Example (Production)

```
[global]
repo1-type=s3
repo1-path=/pgbackrest/acid-ha
repo1-s3-bucket=my-pg-backups
repo1-s3-endpoint=s3.amazonaws.com
repo1-s3-region=eu-central-1
repo1-s3-key=<ACCESS_KEY>
repo1-s3-key-secret=<SECRET_KEY>
repo1-cipher-type=aes-256-cbc
repo1-cipher-pass=<STRONG_PASSPHRASE>
repo1-retention-full=4
repo1-retention-diff=12
```

End of Lab Manual — PostgreSQL Backup & Disaster Recovery with pgBackRest
Practice regularly. An untested backup is not a backup.